МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний університет “Львівська політехніка”

Інститут комп’ютерних наук та інформаційних технологій

Кафедра автоматизованих систем управління

Звіт

про виконання лабораторної роботи №4

з дисципліни “ Паралельні обчислення і розподілені системи”

на тему:

“Мікросервіси”

Виконав:

студент групи ОІ-21СП

Горбатюк Д. О.

Прийняла:

доцентка кафедри АСУ,

Федевич О. Ю.

Львів - 2025

Лабораторна робота №4

Тема: Мікросервіси.

Мета роботи: Оволодіти практичними прийомами розробки мікросервісів.

Завдання лабораторної роботи

Варіант по списку групи - 1

Завдання:

Розбийте систему, розроблену вами у лабораторній роботі №3 на окремі мікросервіси.

ХІД РОБОТИ

1. Спроєктував розбиття роботи із ЛР3 (моноліт) на мікросервіси. Діаграма рішення наведена на Рис. 1.1.

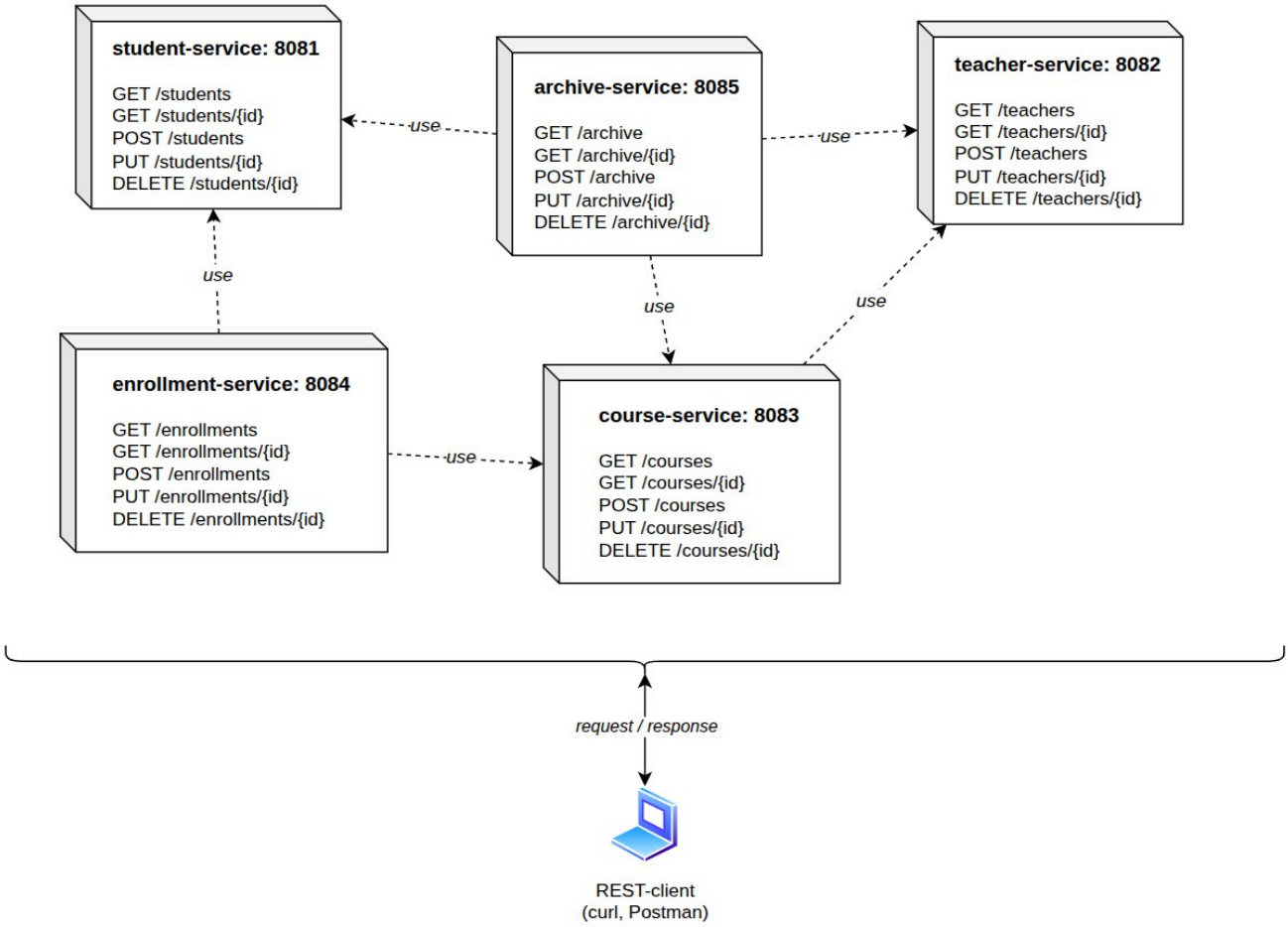

Рис. 1.1. Діаграма рішення у ЛР4

2. Запуск буде відбуватись на різних портах. Рішення зумовлене простотою його виконання, компактністю.
3. Створюємо відповідні проєкти Spring Boot у середовищі Intellij Idea, відповідно до діаграми на Рис. 1.1, проводимо налаштування портів, на яких буде працювати кожен сервіс.
4. Створю перші два сервіси: student-service, teacher-service, адже вони не використовують інших сервісів – запити йдуть тільки до них. Код наведений у додатках XX-XX.

Блок 1 - набір запитів до сервісу student-service;
Блок 2 - відповідь на запит через консоль

```
echo "==============================="
echo "STUDENT SERVICE TEST START"
echo "==============================="

echo
echo "[1] CREATE STUDENT"
curl -X POST http://localhost:8081/students \
  -H "Content-Type: application/json" \
  -d '{
    "fullName": "Danylo Horbatiuk",
    "groupName": "OI-21SP",
    "email": "danylo@student.edu"
  }'

echo
echo
echo "[2] GET ALL STUDENTS"
curl http://localhost:8081/students

echo
echo
echo "[3] GET STUDENT BY ID = 1"
curl http://localhost:8081/students/1

echo
echo
echo "[4] UPDATE STUDENT ID = 1"
curl -X PUT http://localhost:8081/students/1 \
  -H "Content-Type: application/json" \
  -d '{
    "fullName": "Danylo Horbatiuk Updated",
    "groupName": "OI-22SP",
    "email": "danylo.updated@student.edu"
```

```
  }'

echo
echo
echo "[5] GET UPDATED STUDENT BY ID = 1"
curl http://localhost:8081/students/1

echo
echo
echo "[6] DELETE STUDENT ID = 1"
curl -X DELETE http://localhost:8081/students/1

echo
echo
echo "[7] GET ALL STUDENTS AFTER DELETE"
curl http://localhost:8081/students
```

```
==============================
STUDENT SERVICE TEST START
==============================

[1] CREATE STUDENT
{"id":1,"fullName":"Danylo Horbatiuk","groupName":"OI-
21SP","email":"danylo@student.edu"}

[2] GET ALL STUDENTS
[{"id":1,"fullName":"Danylo Horbatiuk","groupName":"OI-
21SP","email":"danylo@student.edu"}]

[3] GET STUDENT BY ID = 1
{"id":1,"fullName":"Danylo Horbatiuk","groupName":"OI-
21SP","email":"danylo@student.edu"}

[4] UPDATE STUDENT ID = 1
{"id":1,"fullName":"Danylo Horbatiuk Updated","groupName":"OI-
22SP","email":"danylo.updated@student.edu"}

[5] GET UPDATED STUDENT BY ID = 1
{"id":1,"fullName":"Danylo Horbatiuk Updated","groupName":"OI-
22SP","email":"danylo.updated@student.edu"}

[6] DELETE STUDENT ID = 1



[7] GET ALL STUDENTS AFTER DELETE
[]
```

Блок 1 - набір запитів до сервісу teacher-service;
Блок 2 - відповідь на запит через консоль

```
echo "==============================="
echo "TEACHER SERVICE TEST START"
echo "==============================="

echo
echo "[1] CREATE TEACHER"
curl -X POST http://localhost:8082/teachers \
  -H "Content-Type: application/json" \
  -d '{
    "fullName": "Ivan Petrenko",
    "department": "Computer Science",
    "email": "ivan.petrenko@university.edu"
  }'

echo
echo
echo "[2] GET ALL TEACHERS"
curl http://localhost:8082/teachers

echo
echo
echo "[3] GET TEACHER BY ID = 1"
curl http://localhost:8082/teachers/1

echo
echo
echo "[4] UPDATE TEACHER ID = 1"
curl -X PUT http://localhost:8082/teachers/1 \
  -H "Content-Type: application/json" \
  -d '{
    "fullName": "Ivan Petrenko Updated",
    "department": "Distributed Systems",
    "email": "ivan.updated@university.edu"
  }'

echo
echo
echo "[5] GET UPDATED TEACHER BY ID = 1"
curl http://localhost:8082/teachers/1

echo
echo
echo "[6] DELETE TEACHER ID = 1"
curl -X DELETE http://localhost:8082/teachers/1

echo
echo
echo "[7] GET ALL TEACHERS AFTER DELETE"
```

```
curl http://localhost:8082/teachers
echo
```

```
==============================
TEACHER SERVICE TEST START
==============================

[1] CREATE TEACHER
{"id":1,"fullName":"Ivan Petrenko","department":"Computer
Science","email":"ivan.petrenko@university.edu"}

[2] GET ALL TEACHERS
[{"id":1,"fullName":"Ivan Petrenko","department":"Computer
Science","email":"ivan.petrenko@university.edu"}]

[3] GET TEACHER BY ID = 1
{"id":1,"fullName":"Ivan Petrenko","department":"Computer
Science","email":"ivan.petrenko@university.edu"}

[4] UPDATE TEACHER ID = 1
{"id":1,"fullName":"Ivan Petrenko Updated","department":"Distributed
Systems","email":"ivan.updated@university.edu"}

[5] GET UPDATED TEACHER BY ID = 1
{"id":1,"fullName":"Ivan Petrenko Updated","department":"Distributed
Systems","email":"ivan.updated@university.edu"}

[6] DELETE TEACHER ID = 1



[7] GET ALL TEACHERS AFTER DELETE
[]
```

5. Далі enrollment-service.

Блок 1 - набір запитів до сервісу enrollment-service;
Блок 2 - відповідь на запит через консоль

```
echo "==============================="
echo "ENROLLMENT SERVICE TEST START"
echo "==============================="

echo
echo "[0] CREATE STUDENT"
curl -X POST http://localhost:8081/students \
  -H "Content-Type: application/json" \
  -d '{
    "fullName": "Danylo Horbatiuk",
```

```
    "groupName": "OI-21SP",
    "email": "danylo@student.edu"
  }'

echo
echo
echo "[1] CREATE TEACHER"
curl -X POST http://localhost:8082/teachers \
  -H "Content-Type: application/json" \
  -d '{
    "fullName": "Ivan Petrenko",
    "department": "Computer Science",
    "email": "ivan.petrenko@university.edu"
  }'

echo
echo
echo "[2] CREATE COURSE"
curl -X POST http://localhost:8083/courses \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Parallel Computing",
    "credits": 5,
    "teacherId": 1
  }'

echo
echo
echo "[3] CREATE ENROLLMENT"
curl -X POST http://localhost:8084/enrollments \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 1,
    "courseId": 1
  }'

echo
echo
echo "[4] GET ALL ENROLLMENTS"
curl http://localhost:8084/enrollments

echo
echo
echo "[5] GET ENROLLMENT BY ID = 1"
curl http://localhost:8084/enrollments/1

echo
echo
echo "[6] NEGATIVE TEST: DUPLICATE ENROLLMENT"
curl -X POST http://localhost:8084/enrollments \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 1,
```

```
    "courseId": 1
  }'

echo
echo
echo "[7] NEGATIVE TEST: INVALID STUDENT"
curl -X POST http://localhost:8084/enrollments \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 99,
    "courseId": 1
  }'

echo
echo
echo "[8] NEGATIVE TEST: INVALID COURSE"
curl -X POST http://localhost:8084/enrollments \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 1,
    "courseId": 99
  }'

echo
echo
echo "[9] UPDATE ENROLLMENT ID = 1"
curl -X PUT http://localhost:8084/enrollments/1 \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 1,
    "courseId": 1
  }'

echo
echo
echo "[10] DELETE ENROLLMENT ID = 1"
curl -X DELETE http://localhost:8084/enrollments/1

echo
echo
echo "[11] GET ALL ENROLLMENTS AFTER DELETE"
curl http://localhost:8084/enrollments

echo
```

```
==============================
ENROLLMENT SERVICE TEST START
==============================

[0] CREATE STUDENT
{"id":1,"fullName":"Danylo Horbatiuk","groupName":"OI-
```

```
21SP","email":"danylo@student.edu"}

[1] CREATE TEACHER
{"id":1,"fullName":"Ivan Petrenko","department":"Computer
Science","email":"ivan.petrenko@university.edu"}

[2] CREATE COURSE
{"id":1,"title":"Parallel Computing","credits":5,"teacherId":1}

[3] CREATE ENROLLMENT
{"id":1,"studentId":1,"courseId":1}

[4] GET ALL ENROLLMENTS
[{"id":1,"studentId":1,"courseId":1}]

[5] GET ENROLLMENT BY ID = 1
{"id":1,"studentId":1,"courseId":1}

[6] NEGATIVE TEST: DUPLICATE ENROLLMENT
This student is already enrolled in this course

[7] NEGATIVE TEST: INVALID STUDENT
Student with id 99 does not exist

[8] NEGATIVE TEST: INVALID COURSE
Course with id 99 does not exist

[9] UPDATE ENROLLMENT ID = 1
{"id":1,"studentId":1,"courseId":1}

[10] DELETE ENROLLMENT ID = 1



[11] GET ALL ENROLLMENTS AFTER DELETE
[]
```

6. Далі archive-service.

Блок 1 - набір запитів до сервісу archive-service;
Блок 2 - відповідь на запит через консоль

```
echo "==============================="
echo "ARCHIVE SERVICE TEST START"
echo "==============================="

echo
echo "[0] CREATE STUDENT"
```

```
curl -X POST http://localhost:8081/students \
  -H "Content-Type: application/json" \
  -d '{
    "fullName": "Danylo Horbatiuk",
    "groupName": "OI-21SP",
    "email": "danylo@student.edu"
  }'

echo
echo
echo "[1] CREATE TEACHER"
curl -X POST http://localhost:8082/teachers \
  -H "Content-Type: application/json" \
  -d '{
    "fullName": "Ivan Petrenko",
    "department": "Computer Science",
    "email": "ivan.petrenko@university.edu"
  }'

echo
echo
echo "[2] CREATE COURSE"
curl -X POST http://localhost:8083/courses \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Parallel Computing",
    "credits": 5,
    "teacherId": 1
  }'

echo
echo
echo "[3] CREATE ARCHIVE RECORD"
curl -X POST http://localhost:8085/archive \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 1,
    "courseId": 1,
    "teacherId": 1,
    "grade": 95
  }'

echo
echo
echo "[4] GET ALL ARCHIVE RECORDS"
curl http://localhost:8085/archive

echo
echo
echo "[5] GET ARCHIVE RECORD BY ID = 1"
curl http://localhost:8085/archive/1

echo
```

```
echo
echo "[6] UPDATE ARCHIVE RECORD ID = 1"
curl -X PUT http://localhost:8085/archive/1 \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 1,
    "courseId": 1,
    "teacherId": 1,
    "grade": 100
  }'

echo
echo
echo "[7] NEGATIVE TEST: INVALID GRADE"
curl -X POST http://localhost:8085/archive \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 1,
    "courseId": 1,
    "teacherId": 1,
    "grade": 150
  }'

echo
echo
echo "[8] NEGATIVE TEST: INVALID STUDENT"
curl -X POST http://localhost:8085/archive \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 99,
    "courseId": 1,
    "teacherId": 1,
    "grade": 90
  }'

echo
echo
echo "[9] NEGATIVE TEST: INVALID TEACHER"
curl -X POST http://localhost:8085/archive \
  -H "Content-Type: application/json" \
  -d '{
    "studentId": 1,
    "courseId": 1,
    "teacherId": 99,
    "grade": 90
  }'

echo
echo
echo "[10] NEGATIVE TEST: INVALID COURSE"
curl -X POST http://localhost:8085/archive \
  -H "Content-Type: application/json" \
  -d '{
```

```
    "studentId": 1,
    "courseId": 99,
    "teacherId": 1,
    "grade": 90
  }'

echo
echo
echo "[11] DELETE ARCHIVE RECORD ID = 1"
curl -X DELETE http://localhost:8085/archive/1

echo
echo
echo "[12] GET ALL ARCHIVE AFTER DELETE"
curl http://localhost:8085/archive

echo
```

```
==============================
ARCHIVE SERVICE TEST START
==============================

[0] CREATE STUDENT
{"id":2,"fullName":"Danylo Horbatiuk","groupName":"OI-
21SP","email":"danylo@student.edu"}

[1] CREATE TEACHER
{"id":2,"fullName":"Ivan Petrenko","department":"Computer
Science","email":"ivan.petrenko@university.edu"}

[2] CREATE COURSE
{"id":2,"title":"Parallel Computing","credits":5,"teacherId":1}

[3] CREATE ARCHIVE RECORD
{"id":1,"studentId":1,"courseId":1,"teacherId":1,"grade":95}

[4] GET ALL ARCHIVE RECORDS
[{"id":1,"studentId":1,"courseId":1,"teacherId":1,"grade":95}]

[5] GET ARCHIVE RECORD BY ID = 1
{"id":1,"studentId":1,"courseId":1,"teacherId":1,"grade":95}

[6] UPDATE ARCHIVE RECORD ID = 1
{"id":1,"studentId":1,"courseId":1,"teacherId":1,"grade":100}

[7] NEGATIVE TEST: INVALID GRADE
Grade must be between 0 and 100

[8] NEGATIVE TEST: INVALID STUDENT
Student with id 99 does not exist
```

```
[9] NEGATIVE TEST: INVALID TEACHER
Teacher with id 99 does not exist

[10] NEGATIVE TEST: INVALID COURSE
Course with id 99 does not exist

[11] DELETE ARCHIVE RECORD ID = 1



[12] GET ALL ARCHIVE AFTER DELETE
[]
```

Отже, відповідну структуру реалізовано. Проте є великі недоліки використання саме такої схеми мікросервісів, адже кожен із них модерується і обслуговується як окремий проєкт, що створює труднощі у підтримці їх життєздатності.

ВИСНОВКИ

У ході роботи монолітну систему з лабораторної роботи №3 було розбито на окремі мікросервіси, які запускаються на різних портах і взаємодіють між собою через HTTP-запити. Було реалізовано student-service, teacher-service, course-service, enrollment-service та archive-service.

Загалом робота допомогла краще зрозуміти принцип побудови мікросервісної архітектури. Разом із тим, стало видно, що така схема ускладнює підтримку системи, бо кожен сервіс потрібно окремо запускати, налаштовувати і контролювати.

ДОДАТКИ

Додаток 1

Код файлу ArchiveController.java

```
package dev.horbatiuk.pcds.lab3.controller;

import dev.horbatiuk.pcds.lab3.model.ArchiveRecord;
import dev.horbatiuk.pcds.lab3.service.ArchiveService;
import dev.horbatiuk.pcds.lab3.service.CourseService;
import dev.horbatiuk.pcds.lab3.service.StudentService;
import dev.horbatiuk.pcds.lab3.service.TeacherService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/archive")
public class ArchiveController {
    private final ArchiveService archiveService;
    private final StudentService studentService;
    private final CourseService courseService;
    private final TeacherService teacherService;

    public ArchiveController(ArchiveService archiveService,
                             StudentService studentService,
                             CourseService courseService,
                             TeacherService teacherService) {
        this.archiveService = archiveService;
        this.studentService = studentService;
        this.courseService = courseService;
        this.teacherService = teacherService;
    }

    @GetMapping
    public List<ArchiveRecord> getAll() {
        return archiveService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<ArchiveRecord> getById(@PathVariable Long id) {
        ArchiveRecord record = archiveService.getById(id);
        return record != null ? ResponseEntity.ok(record) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<?> create(@RequestBody ArchiveRecord record) {
        if (!studentService.exists(record.getStudentId())) {
            return ResponseEntity.badRequest().body("Student with given id does
not exist");
```

```
        }
        if (!courseService.exists(record.getCourseId())) {
            return ResponseEntity.badRequest().body("Course with given id does not
exist");
        }
        if (!teacherService.exists(record.getTeacherId())) {
            return ResponseEntity.badRequest().body("Teacher with given id does
not exist");
        }
        if (record.getGrade() == null || record.getGrade() < 0 ||
record.getGrade() > 100) {
            return ResponseEntity.badRequest().body("Grade must be in range
0..100");
        }
        return ResponseEntity.ok(archiveService.create(record));
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> update(@PathVariable Long id, @RequestBody
ArchiveRecord record) {
        if (!studentService.exists(record.getStudentId())) {
            return ResponseEntity.badRequest().body("Student with given id does
not exist");
        }
        if (!courseService.exists(record.getCourseId())) {
            return ResponseEntity.badRequest().body("Course with given id does not
exist");
        }
        if (!teacherService.exists(record.getTeacherId())) {
            return ResponseEntity.badRequest().body("Teacher with given id does
not exist");
        }
        if (record.getGrade() == null || record.getGrade() < 0 ||
record.getGrade() > 100) {
            return ResponseEntity.badRequest().body("Grade must be in range
0..100");
        }
        ArchiveRecord updated = archiveService.update(id, record);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return archiveService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
}
```

Додаток 2

Код файлу CourseController.java

```
package dev.horbatiuk.pcds.lab3.controller;

import dev.horbatiuk.pcds.lab3.model.Course;
import dev.horbatiuk.pcds.lab3.service.CourseService;
import dev.horbatiuk.pcds.lab3.service.TeacherService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/courses")
public class CourseController {
    private final CourseService courseService;
    private final TeacherService teacherService;

    public CourseController(CourseService courseService, TeacherService
teacherService) {
        this.courseService = courseService;
        this.teacherService = teacherService;
    }

    @GetMapping
    public List<Course> getAll() {
        return courseService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Course> getById(@PathVariable Long id) {
        Course course = courseService.getById(id);
        return course != null ? ResponseEntity.ok(course) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<?> create(@RequestBody Course course) {
        if (!teacherService.exists(course.getTeacherId())) {
            return ResponseEntity.badRequest().body("Teacher with given id does
not exist");
        }
        return ResponseEntity.ok(courseService.create(course));
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> update(@PathVariable Long id, @RequestBody Course
course) {
        if (!teacherService.exists(course.getTeacherId())) {
```

```
            return ResponseEntity.badRequest().body("Teacher with given id does
not exist");
        }
        Course updated = courseService.update(id, course);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return courseService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
}
```

Додаток 3

Код файлу EnrollmentController.java

```
package dev.horbatiuk.pcds.lab3.controller;

import dev.horbatiuk.pcds.lab3.model.Enrollment;
import dev.horbatiuk.pcds.lab3.service.CourseService;
import dev.horbatiuk.pcds.lab3.service.EnrollmentService;
import dev.horbatiuk.pcds.lab3.service.StudentService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/enrollments")
public class EnrollmentController {
    private final EnrollmentService enrollmentService;
    private final StudentService studentService;
    private final CourseService courseService;

    public EnrollmentController(EnrollmentService enrollmentService,
                                StudentService studentService,
                                CourseService courseService) {
        this.enrollmentService = enrollmentService;
        this.studentService = studentService;
        this.courseService = courseService;
    }

    @GetMapping
    public List<Enrollment> getAll() {
        return enrollmentService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Enrollment> getById(@PathVariable Long id) {
        Enrollment enrollment = enrollmentService.getById(id);
        return enrollment != null ? ResponseEntity.ok(enrollment) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<?> create(@RequestBody Enrollment enrollment) {
        if (!studentService.exists(enrollment.getStudentId())) {
            return ResponseEntity.badRequest().body("Student with given id does
not exist");
        }
        if (!courseService.exists(enrollment.getCourseId())) {
            return ResponseEntity.badRequest().body("Course with given id does not
exist");
        }
```

```
        if (enrollmentService.isStudentAlreadyEnrolled(enrollment.getStudentId(),
enrollment.getCourseId())) {
            return ResponseEntity.badRequest().body("Student is already enrolled
in this course");
        }
        return ResponseEntity.ok(enrollmentService.create(enrollment));
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> update(@PathVariable Long id, @RequestBody Enrollment
enrollment) {
        if (!studentService.exists(enrollment.getStudentId())) {
            return ResponseEntity.badRequest().body("Student with given id does
not exist");
        }
        if (!courseService.exists(enrollment.getCourseId())) {
            return ResponseEntity.badRequest().body("Course with given id does not
exist");
        }
        Enrollment updated = enrollmentService.update(id, enrollment);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return enrollmentService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
}
```

Додаток 4

Код файлу StudentController.java

```
package dev.horbatiuk.pcds.lab3.controller;

import dev.horbatiuk.pcds.lab3.model.Student;
import dev.horbatiuk.pcds.lab3.service.StudentService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {
    private final StudentService studentService;

    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @GetMapping
    public List<Student> getAll() {
        return studentService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getById(@PathVariable Long id) {
        Student student = studentService.getById(id);
        return student != null ? ResponseEntity.ok(student) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<Student> create(@RequestBody Student student) {
        return ResponseEntity.ok(studentService.create(student));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Student> update(@PathVariable Long id, @RequestBody
Student student) {
        Student updated = studentService.update(id, student);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return studentService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
```

}

Додаток 5

Код файлу TeacherController.java

```
package dev.horbatiuk.pcds.lab3.controller;

import dev.horbatiuk.pcds.lab3.model.Teacher;
import dev.horbatiuk.pcds.lab3.service.TeacherService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/teachers")
public class TeacherController {
    private final TeacherService teacherService;

    public TeacherController(TeacherService teacherService) {
        this.teacherService = teacherService;
    }

    @GetMapping
    public List<Teacher> getAll() {
        return teacherService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Teacher> getById(@PathVariable Long id) {
        Teacher teacher = teacherService.getById(id);
        return teacher != null ? ResponseEntity.ok(teacher) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<Teacher> create(@RequestBody Teacher teacher) {
        return ResponseEntity.ok(teacherService.create(teacher));
    }
    @PutMapping("/{id}")
    public ResponseEntity<Teacher> update(@PathVariable Long id, @RequestBody
Teacher teacher) {
        Teacher updated = teacherService.update(id, teacher);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return teacherService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
}
```

Додаток 6

Код файлу ArchiveRecord.java

```
package dev.horbatiuk.pcds.lab3.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class ArchiveRecord {
    private Long id;
    private Long studentId;
    private Long courseId;
    private Long teacherId;
    private Integer grade;
    private String completedAt;
}
```

Додаток 7

Код файлу Course.java

```
package dev.horbatiuk.pcds.lab3.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class Course {
    private Long id;
    private String name;
    private String description;
    private Integer credits;
    private Long teacherId;
}
```

Додаток 8

Код файлу Enrollment.java

```
package dev.horbatiuk.pcds.lab3.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
```

```
@NoArgsConstructor
public class Enrollment {
    private Long id;
    private Long studentId;
    private Long courseId;
    private String enrollmentDate;
    private String status;
}
```

Додаток 9

Код файлу Student.java

```
package dev.horbatiuk.pcds.lab3.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class Student {
    private Long id;
    private String fullName;
    private String groupName;
    private String email;
}
```

Додаток 10

Код файлу Student.java

```
package dev.horbatiuk.pcds.lab3.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class Teacher {
    private Long id;
    private String fullName;
    private String department;
    private String email;
}
```

Додаток 11

Код файлу ArchiveService.java

```
package dev.horbatiuk.pcds.lab3.service;

import dev.horbatiuk.pcds.lab3.model.ArchiveRecord;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class ArchiveService {
    private final Map<Long, ArchiveRecord> archive = new LinkedHashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<ArchiveRecord> getAll() {
        return new ArrayList<>(archive.values());
    }

    public ArchiveRecord getById(Long id) {
        return archive.get(id);
    }

    public ArchiveRecord create(ArchiveRecord record) {
        long id = counter.getAndIncrement();
        record.setId(id);
        archive.put(id, record);
        return record;
    }

    public ArchiveRecord update(Long id, ArchiveRecord record) {
        if (!archive.containsKey(id)) {
            return null;
        }
        record.setId(id);
        archive.put(id, record);
        return record;
    }

    public boolean delete(Long id) {
        return archive.remove(id) != null;
    }
}
```

Додаток 12

Код файлу CourseService.java

```
package dev.horbatiuk.pcds.lab3.service;

import dev.horbatiuk.pcds.lab3.model.Course;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class CourseService {
    private final Map<Long, Course> courses = new LinkedHashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Course> getAll() {
        return new ArrayList<>(courses.values());
    }

    public Course getById(Long id) {
        return courses.get(id);
    }

    public Course create(Course course) {
        long id = counter.getAndIncrement();
        course.setId(id);
        courses.put(id, course);
        return course;
    }

    public Course update(Long id, Course course) {
        if (!courses.containsKey(id)) {
            return null;
        }
        course.setId(id);
        courses.put(id, course);
        return course;
    }

    public boolean delete(Long id) {
        return courses.remove(id) != null;
    }

    public boolean exists(Long id) {
        return courses.containsKey(id);
    }
}
```

Додаток 13

Код файлу EnrollmentService.java

```
package dev.horbatiuk.pcds.lab3.service;

import dev.horbatiuk.pcds.lab3.model.Enrollment;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class EnrollmentService {
    private final Map<Long, Enrollment> enrollments = new LinkedHashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Enrollment> getAll() {
        return new ArrayList<>(enrollments.values());
    }
    public Enrollment getById(Long id) {
        return enrollments.get(id);
    }
    public Enrollment create(Enrollment enrollment) {
        long id = counter.getAndIncrement();
        enrollment.setId(id);
        enrollments.put(id, enrollment);
        return enrollment;
    }
    public Enrollment update(Long id, Enrollment enrollment) {
        if (!enrollments.containsKey(id)) {
            return null;
        }
        enrollment.setId(id);
        enrollments.put(id, enrollment);
        return enrollment;
    }
    public boolean delete(Long id) {
        return enrollments.remove(id) != null;
    }
    public boolean exists(Long id) {
        return enrollments.containsKey(id);
    }
    public boolean isStudentAlreadyEnrolled(Long studentId, Long courseId) {
        return enrollments.values().stream()
                .anyMatch(e -> e.getStudentId().equals(studentId) &&
e.getCourseId().equals(courseId));
    }
}
```

Додаток 14

Код файлу StudentService.java

```
package dev.horbatiuk.pcds.lab3.service;

import dev.horbatiuk.pcds.lab3.model.Student;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class StudentService {
    private final Map<Long, Student> students = new LinkedHashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Student> getAll() {
        return new ArrayList<>(students.values());
    }

    public Student getById(Long id) {
        return students.get(id);
    }

    public Student create(Student student) {
        long id = counter.getAndIncrement();
        student.setId(id);
        students.put(id, student);
        return student;
    }

    public Student update(Long id, Student student) {
        if (!students.containsKey(id)) {
            return null;
        }
        student.setId(id);
        students.put(id, student);
        return student;
    }

    public boolean delete(Long id) {
        return students.remove(id) != null;
    }
    public boolean exists(Long id) {
        return students.containsKey(id);
    }
}
```

Додаток 15

Код файлу TeacherService.java

```
package dev.horbatiuk.pcds.lab3.service;

import dev.horbatiuk.pcds.lab3.model.Teacher;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class TeacherService {
    private final Map<Long, Teacher> teachers = new LinkedHashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Teacher> getAll() {
        return new ArrayList<>(teachers.values());
    }

    public Teacher getById(Long id) {
        return teachers.get(id);
    }

    public Teacher create(Teacher teacher) {
        long id = counter.getAndIncrement();
        teacher.setId(id);
        teachers.put(id, teacher);
        return teacher;
    }

    public Teacher update(Long id, Teacher teacher) {
        if (!teachers.containsKey(id)) {
            return null;
        }
        teacher.setId(id);
        teachers.put(id, teacher);
        return teacher;
    }

    public boolean delete(Long id) {
        return teachers.remove(id) != null;
    }
    public boolean exists(Long id) {
        return teachers.containsKey(id);
    }
}
```

Додаток 16

Код файлу ElectiveApplication.java

```
package dev.horbatiuk.pcds.lab3;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ElectiveApplication {
    public static void main(String[] args) {
        SpringApplication.run(ElectiveApplication.class, args);
    }
}
```